天津大學

Tianjin University

TJU Homework Making

题目图片自动排版成 PDF

使用文档

安装 · GUI · CLI · 配置 · 流水线 · 排错 · 构建

项目版本 0.1.0	推荐平台 Windows 11 / Python 3.11+
验证结果 29 passed, 1 skipped	推荐入口 launch_gui.bat / dist EXE
最终输出 A4 PDF	生成日期

依据当前本地项目源码、测试、构建脚本与实际界面生成

A 目录

阅读建议	目录保留一级章节，详细小节可通过 PDF 书签直接跳转。

1	快速开始	3	7	项目结构与模块关系	9
2	GUI 界面与基础操作	4	8	处理流水线	10
3	完整 GUI 工作流	5	9	裁剪、排版与输出	11
4	版式与高级工具	6	10	输出文件与排错	12
5	CLI 使用	7	11	开发、测试与打包	13
6	配置文件	8	A	文件盘点与索引	14

文档范围 从安装、GUI 与 CLI 使用，到配置、处理流水线、项目结构、调试与打包。内容依据当前工作区 3,011 个文件的目录盘点，以及源码、测试、构建脚本和实际界面核验。

适用对象	需要把题目截图、扫描件或 PDF 自动切题并统一排版的学生、教师与资料整理者。

1 1. 快速开始

1.1 选择启动方式

Windows 用户优先使用安装包、当前 dist 中的 EXE，或根目录的 launch_gui.bat。

方式	入口	适用情况
安装包	dist_installer\pdf_exam_formatter_setup.exe	希望创建开始菜单或桌面入口的普通用户。
免安装 EXE	dist\pdf_exam_formatter_gui\pdf_exam_formatter_gui.exe	项目目录完整，直接运行打包版。
统一启动脚本	launch_gui.bat	优先启动打包 EXE；若不存在，再尝试本机 Python。
源码运行	python -m app.gui	开发、调试或修改源码。

路径提醒 根目录两个 .lnk 快捷方式仍指向旧目录或多了一层 pdf_exam_formatter，可能无法启动。请以 launch_gui.bat 或当前 dist EXE 为准。

1.2 第一次运行

1. 添加图片 / PDF
2. 设置输出与参数
3. 开始处理
4. 预览并导出

- 点击“添加文件”，或拖拽/粘贴一个或多个图片、PDF。输入按列表顺序处理。
- 选择输出目录并填写 PDF 文件名；未写 .pdf 时会自动补齐。
- 一般保持“标准”、DPI 220、每页 4 题；需要时填写学号、班级、姓名。
- 点击“开始处理”。完成后预览输出，必要时手动校正，再打开输出目录。

1.3 源码环境

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
python -m app.gui
```

2 2. GUI 界面与基础操作

2.1 主界面

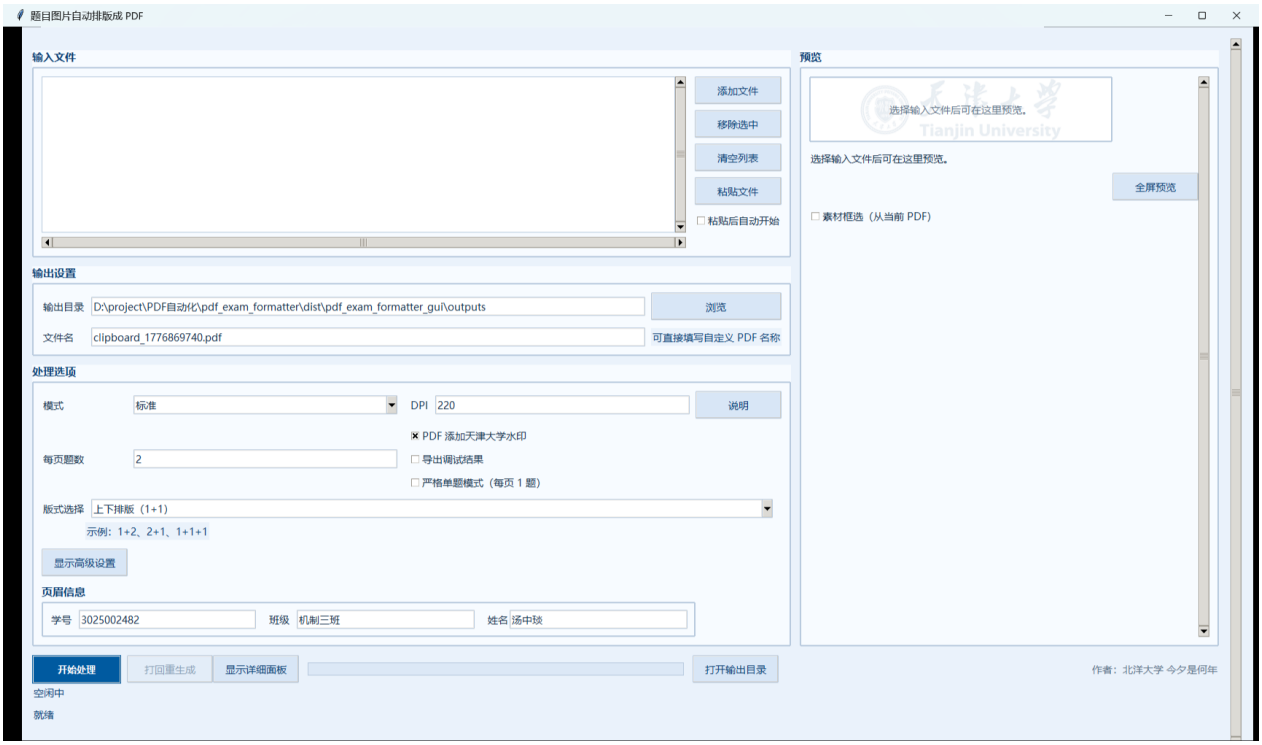


图 1 当前项目 GUI 实际运行界面；截图用于定位功能区域。

编号	区域	用途
1	输入文件	添加、移除、清空、粘贴；支持多选、拖拽与横向滚动。
2	预览	查看当前输入或结果；可全屏，也可进入 PDF 素材框选。
3	输出设置	分别设置输出目录和文件名。
4	处理选项	模式、DPI、题数、版式、水印、debug、严格单题与高级设置。
5	执行区	开始处理、打回重生成、进度、输出目录与详细面板。

2.2 常用默认值

选项	默认值	说明
模式	标准 / normal	完整执行检测、裁剪和排版。
DPI	220	清晰度与速度的均衡值。
每页题数	4	默认 2×2；其他题数自动生成均衡版式。
水印	开启	使用 assets/branding/tju_logo_source.png。
Debug	关闭	打开后导出检测框、裁剪表和渲染预览。

3 3. 完整 GUI 工作流

3.1 准备输入

输入方式	行为
添加文件	支持 .png/.jpg/.jpeg/.bmp/.tif/.tiff/.webp/.pdf，可一次选择多个。
拖拽	Windows 下可直接拖进窗口；任务运行时会忽略新的拖入。
粘贴	读取剪贴板文件列表、file URI、路径文本，也能接收剪贴板图片。
粘贴后自动开始	启用后，粘贴到输入列表即可立即触发处理。
处理顺序	输入列表顺序就是处理顺序。图片各生成一个源页面；PDF 先按页光栅化，再与图片走同一条检测流程。

3.2 设置输出

- 输出目录不存在时会自动创建；文件名中的 / 和 \ 会替换为下划线。
- 处理完成后，“打开输出目录”会定位到最终 PDF 所在文件夹。
- GUI 会把最近输入、参数与窗口状态保存到 .cache/gui_state.json。

3.3 选择处理参数

功能	什么时候用	关键影响
快速模式	批量预览或只做轻量处理	只减少次要精修/调试，不删除页眉、外框与版式结构。
DPI 180–200	希望更快	速度更快，但小字与细线保留能力下降。
DPI 240–300	小字、细图、模糊扫描	更清晰，但更慢且占用更多内存。
严格单题模式	带题号 PDF 或每页 1 题	每页题数强制为 1，并启用题号文本分段/投影兜底。
导出调试结果	检测框不准或要复盘	写入 debug\<输出名>\。
天津大学水印	需要统一校内视觉	在题目网格后方绘制低透明度校徽与中英文校名。

3.4 运行、预览与复核

- 进度依次显示：读取输入、解析模板、检测小题、安全裁剪、页面排版、生成 PDF、导出调试结果。
- 成功后日志会给出源页面数、检测题数、输出页数、输出路径和 debug 路径。
- 重点检查题号、图形边缘、最后一行文字，以及题目之间是否发生误并或漏裁。

4 4. 版式与高级工具

4.1 每页题数与版式

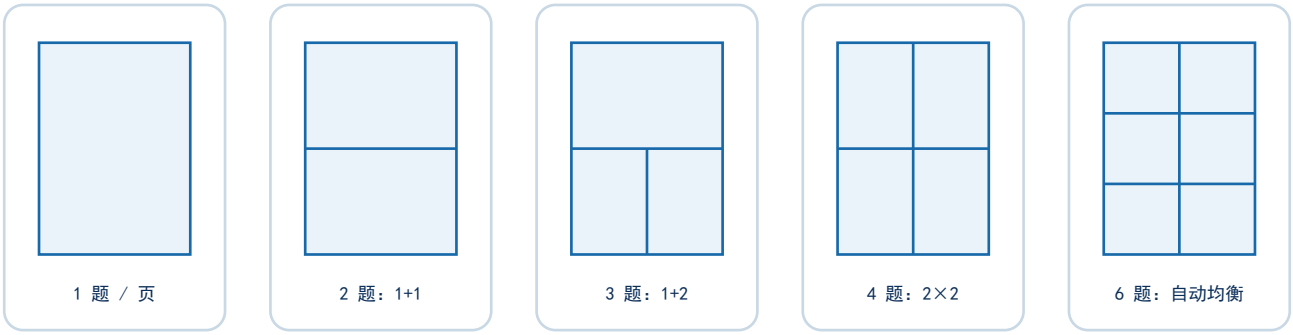


图 2 无模板时按题数动态生成版式；2 题和 3 题可在 GUI 中选择预设或自定义行模式。

模板锁定题数	勾选：模板网格决定每页容量；取消：保留模板页面几何，但按“每页题数”重新生成动态网格。

4.2 高级功能速查

高级功能	入口 / 行为	产物或限制
版式模板	高级设置 -> 选择 PDF/图片；自动识别外框、页眉带和分栏。	锁定题数时完全采用模板网格；取消后只沿用模板几何风格。
素材框选	预览区勾选“素材框选（从当前 PDF）”，拖框保存题图。	保存到 .cache\selected_materials\，并自动加入输入列表。
手动校正	处理完成后显示面板；拖动红框四边，或按像素步长精调。	可重渲染当前页，并直接覆盖导出到设置的 PDF 路径。
打回重生成	处理成功后可依次尝试保守分题、强力防误并、按题带兜底。	每次基于原配置调整检测阈值，不需要手工编辑配置文件。
全局配置	加载 JSON/YAML，覆盖检测、裁剪、布局、字体、水印等。	适合长期固定参数。
页眉配置	单独加载 JSON/YAML 页眉设置。	不改变题目检测与主体布局。

4.3 手动校正建议

- 先用“聚焦题目 / 查看整页”切换视图，再以 5 px 左右步长调整。
- 红框对应当前题目的裁剪边界；调整后先重渲染当前页，确认题号、箭头和底部注释完整。
- 手动校正只在已有处理结果上工作；重新执行完整流水线会重新生成自动裁剪。

5 5. CLI 使用

5.1 基本命令

```
python -m app.main ^
 --input .\input1.png .\input2.pdf ^
 --output .\outputs\result.pdf ^
 --dpi 220 ^
 --per-page 4 ^
 --watermark true
```

严格单题与身份页眉示例：

```
python -m app.main ^
 --input .\exam.pdf ^
 --output .\outputs\single.pdf ^
 --strict-single-question true ^
 --student-id 3025002482 ^
 --class-name "机制三班" ^
 --student-name "张三"
```

5.2 参数速查

参数	用途	示例 / 取值
--input	一个或多个图片/PDF 输入	a.png b.pdf
--output	输出 PDF 路径	.\outputs\result.pdf
--mode	处理模式	normal / fast
--debug / --watermark	调试产物 / 天津大学水印	true / false
--dpi	图片与 PDF 光栅化 DPI	220
--per-page	每页题数	1、2、3、4、6…
--strict-single-question	严格单题模式	true / false
--layout-template	版式模板 PDF/图片	template.pdf
--template-lock-per-page	模板是否锁定容量	true / false
--config / --header-config	全局 / 页眉 JSON 或 YAML	config.yaml / header.yaml
--student-id / --class-name / --student-name	页眉身份信息	字符串
--print-config	打印最终生效配置并退出	无值开关
--gui	从 CLI 入口启动 GUI	无值开关

6 6. 配置文件

6.1 覆盖顺序

优先级	内置默认值 -> --config 全局文件 -> --header-config 页眉文件 -> CLI/GUI 当前输入。合并为递归 deep merge，不会因为只改一个字段而清空同组其他字段。

6.2 YAML 示例

```yaml
io:
 image_dpi: 240
 pdf_dpi: 240
layout:
 problems_per_page: 4
 template_lock_per_page: true
watermark:
 enabled: true
header:
 style: reference_identity
 identity:
 student_id: "3025002482"
 class_name: "机制三班"
 student_name: "张三"
debug:
 enabled: true
```

查看最终生效配置：

```
python -m app.main --input .\input.png --output .\outputs\result.pdf --config .\config.yaml --print-config
```

6.3 配置组速查

配置组	内容
io	图片/PDF DPI、输出目录、debug 目录、缓存目录、支持格式。
detection	二值化、连通域、合并阈值、网格检测、PDF 题号检测、严格模式。
crop	扩框、保底边距、去白边阈值、稀疏/高/宽/圆形题的保护策略。
layout	A4、每页题数、行模式、模板、边距、页眉位置、留白比例。
fonts / header	字体搜索与回退；身份页眉或旧版标题+字段页眉。
watermark	素材路径、颜色、透明度、宽高比例、垂直位置。
debug / performance	调试产物开关、预览页数、normal/fast 行为。

字体回退	渲染器依次尝试配置字体、Windows 字体目录与 assets/fonts，最后回退到 STSong-Light。若要固定版式，可把字体文件放到 assets/fonts 并写入 user_font_paths。

7 7. 项目结构与模块关系

7.1 目录层级

```
TJU-Homework-Making/
├── .github/           CI、Issue 与 PR 模板
├── app/               核心源码、GUI 与 CLI
├── assets/            水印、字体与示例素材
├── docs/              完整使用文档
├── tests/             单元与回归测试
├── pyproject.toml     元数据、依赖、入口与 pytest
├── requirements*.txt 安装兼容入口
├── build_*.bat        Windows 构建脚本
└── *.spec / *.iss     PyInstaller 与 Inno Setup
```

7.2 模块职责

层级	模块	职责
入口	gui.py / main.py / __main__.py	Tkinter GUI、CLI 参数、输入验证与安装入口。
编排	pipeline.py	串联读取、检测、裁剪、布局、渲染和 debug。
数据 / 配置	models.py / config.py	跨阶段 dataclass；默认值、JSON/YAML 与覆盖合并。
视觉检测	preprocess.py / detect_regions.py / refine_regions.py	二值化、连通域、题号、网格、合并、去重和阅读顺序。
裁剪 / 布局	cropper.py / layout_engine.py / template_layout.py	安全扩框、去白边、动态网格与模板推断。
输出	header_template.py / pdf_renderer.py	页眉、水印、网格和 ReportLab PDF。
辅助	io_utils.py / debug_export.py / runtime_paths.py	Windows 安全读写、调试产物、源码/安装/冻结路径解析。

仓库边界	源码、测试、资源、文档和配置进入 Git；缓存、debug、输出及 build/dist 等构建产物由 .gitignore 排除。

8 8. 处理流水线

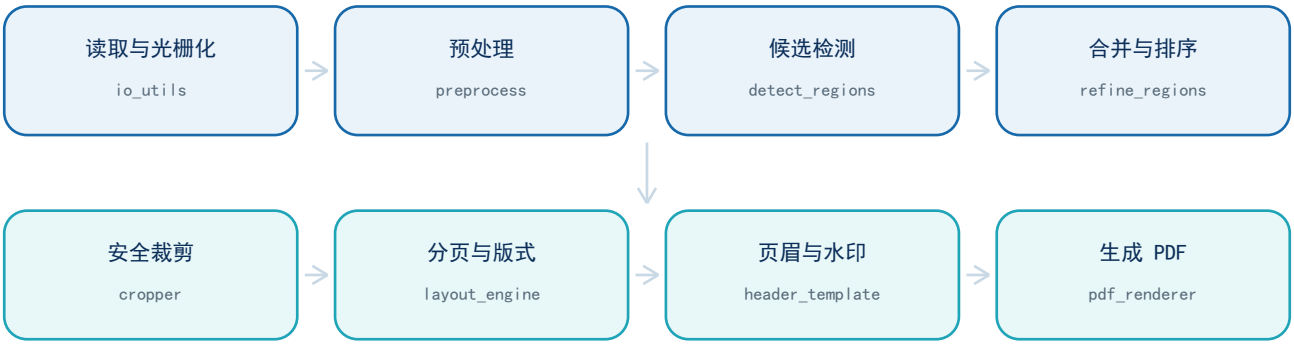
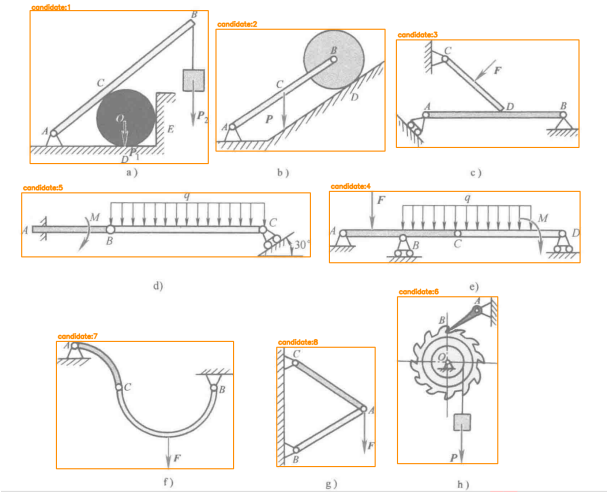


图 3 run_pipeline 主路径；template_layout 在排版前解析可选模板，debug_export 在渲染后导出复盘素材。

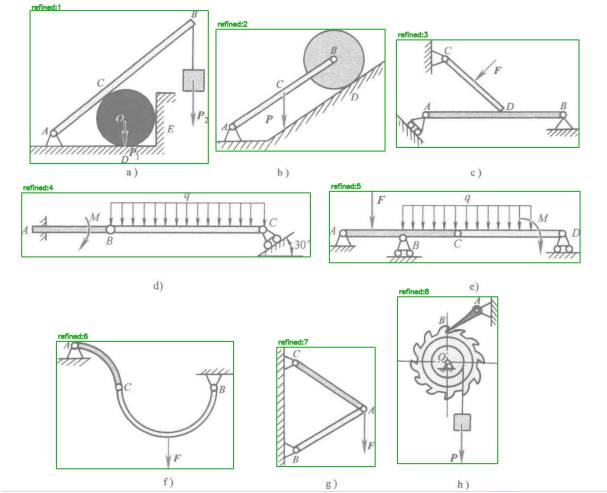
8.1 检测策略

● 可选文本 PDF：优先从题号行推断题目跨度，并对底部稀疏噪声做裁切。	● 严格模式图片：可用水平投影按题带分段，作为连通域之外的兜底。
● 通用路径：灰度、轻度高斯模糊、Otsu/自适应阈值、反色与闭运算，再提取连通域。	● 稳定 2×2 网格会按单元格限制合并，避免跨栏误并；最终过滤、去重、合并并排序。

8.2 候选框与精修框



候选检测：橙色框来自初始连通域 / 题号检测



精修结果：绿色框完成合并、去重与阅读顺序整理

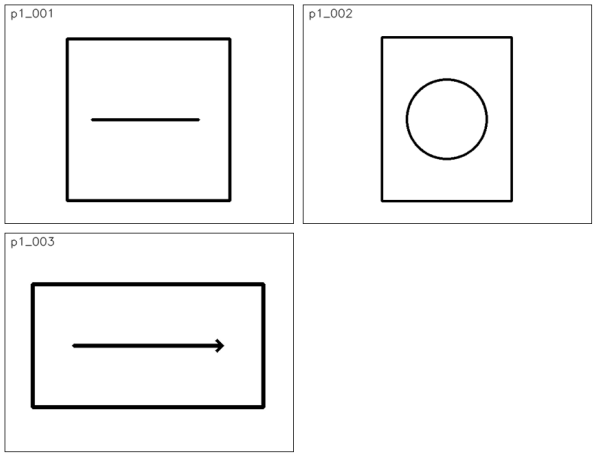
9 9. 裁剪、排版与输出

9.1 安全裁剪

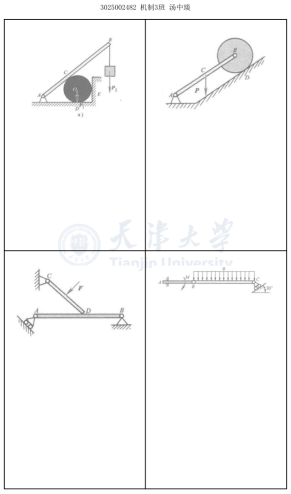
- 检测框按左右、上下不对称比例扩张；底部保留更多空间，以保护注释、箭头和最后一行。
- 去白边采用保守策略：先膨胀细线，再按内容框保留最小边距；改善不足 3% 时保留原裁剪。
- 高、宽、近圆形和稀疏题图使用不同的边距增益，降低图形边缘被切掉的风险。

9.2 页面布局

- 输出固定为 A4 竖版；题图放在单元格上部，默认留出约 50%–67% 的书写空间。
- 最后一页仍保留空单元格与边框，保证整份作业版式稳定。
- 模板模式可直接使用检测到的外框和行列边界，也可只借用页面几何后重新动态分页。



裁剪 contact sheet：核对每个题块是否完整



最终输出：A4 网格、身份页眉与天津大学水印

输出检查	至少抽查第一页、最后一页和包含大图 / 长题干的页面；若只有个别边界不准，优先手动校正。

10 10. 输出文件与排错

10.1 运行产物

路径	内容	是否建议保留
outputs	项目历史示例和回归输出 PDF。	按需
debug\<输出名>\detection_candidates	候选框叠加图。	排错时保留
debug\<输出名>\detection_refined	精修框叠加图。	排错时保留
debug\<输出名>\crop_contact_sheet.png	全部裁剪题图拼版。	最适合快速检查漏裁
debug\<输出名>\render_preview	最终 PDF 前几页 PNG。	排版排错时保留
.cache\gui_state.json	GUI 最近设置与输入路径。	自动管理
.cache\selected_materials	PDF 框选生成的图片输入。	确认不再使用后再清理
.cache\manual_preview_page.pdf	手动校正当前页的临时预览。	自动覆盖

10.2 常见问题

现象	判断顺序	处理方法
启动失败	确认使用当前路径下 launch_gui.bat / dist EXE。	不要依赖两个旧 .lnk；源码方式确认 Python 3.11+ 与依赖。
完全检测不到	查看 refined 图是否为空。	程序会整页兜底；可改 DPI、打开严格模式或使用素材框选。
两题被并成一题	候选正确但 refined 合并。	使用“打回重生成”的保守分题 / 强力防误并，或调低合并阈值。
题目底部被切掉	查看 contact sheet。	先手动下移底边界；长期出现时提高 crop.expand_bottom_ratio。
模板识别失败	模板是否有清晰连续外框与分栏线。	提高模板 DPI，换更干净参考页，或取消模板改用动态布局。
中文字体异常	确认字体搜索目录与文件格式。	把 TTF/OTF/TTC 放入 assets/fonts，或配置 user_font_paths。
处理很慢 / 内存高	检查输入页数和 DPI。	降低到 180–200，使用 fast，关闭 debug。

10.3 建议排查顺序

01	02	03	04	05
输入原图	候选框	精修框	裁剪拼版	渲染预览
确认分辨率、页序与旋转	判断是否检测到题块	判断是否误并或漏并	检查边缘与最后一行	检查网格、页眉与水印

处理原则	候选框错误时优先重跑检测；只有少量裁剪边界不准时，优先使用手动校正，避免重复完整流水线。

11 11. 开发、测试与打包

11.1 提交前验证

```
python -m compileall -q app
python -m pytest
python -m app.main --version
python -m app --version
python -m pip wheel --no-deps --wheel-dir tmp\wheel-check .
```

本次核验	31 passed, 1 skipped；CLI 两种模块入口、TOML/YAML、文档链接、版本一致性和本地 wheel 构建均通过。

11.2 GitHub 与项目元数据

文件 / 目录	作用
pyproject.toml	项目元数据、Python 版本、依赖、CLI/GUI 入口和 pytest 配置。
requirements.txt / requirements-dev.txt	兼容 pip 工作流的运行与开发环境入口。
.github/workflows/ci.yml	Windows 上使用 Python 3.11/3.12 编译、测试、CLI 与 wheel 校验。
CONTRIBUTING / SECURITY / CODE_OF_CONDUCT	贡献流程、私密漏洞报告与协作行为规范。
ISSUE_TEMPLATE / PULL_REQUEST_TEMPLATE	统一问题描述、隐私提示和提交前检查项。

11.3 Windows 打包

build_exe.bat 按 pdf_exam_formatter_gui.spec 生成免安装 GUI；build_setup.bat 再按 pdf_exam_formatter_setup.iss 生成安装包。构建目录 dist、build 与 dist_installer 不进入 Git。

CI 边界	CI 验证源码安装和 wheel，不在每次提交中构建 Windows EXE 或安装包；发布前再执行完整打包验收。

A 附录 A：提交前文件索引

附录 A.1 根目录规范文件

文件	作用
README.md / docs/	项目首页与图文使用文档。
pyproject.toml	版本、依赖、入口、构建和测试元数据。
requirements*.txt	运行与开发环境的 pip 安装入口。
.gitignore / .gitattributes / .editorconfig	忽略产物、统一换行与编辑器格式。
LICENSE / CHANGELOG.md	MIT License 与版本更新记录。
CONTRIBUTING / SECURITY / CODE_OF_CONDUCT	贡献、安全与社区协作规范。
build_*.bat / *.spec / *.iss	Windows 免安装程序和安装包构建。

附录 A.2 关键实现文件索引

功能	首先阅读	功能	首先阅读
修改 GUI	app/gui.py	新增 CLI 参数	app/main.py + app/config.py
调整分题	app/detect_regions.py + app/refine_regions.py	调整裁剪保护	app/cropper.py
调整版式	app/layout_engine.py + app/template_layout.py	页眉 / 水印 / PDF	app/header_template.py + app/pdf_renderer.py
debug 产物	app/debug_export.py	改变总流程	app/pipeline.py
运行路径	app/runtime_paths.py	版本与入口	app/__init__.py + app/__main__.py

附录 A.3 Git 提交范围

进入 Git	本地保留但忽略
app、tests、assets、docs、.github	.cache、debug、outputs、output、tmp
README、元数据、规范文件、构建脚本	build、dist、dist_installer、*.egg-info、__pycache__

一句话理解项目	一个以传统视觉为主、模板与手动校正为辅的 Windows 题目整理工具；仓库层面已具备安装元数据、CI、测试、社区规范和完整文档。